SEMANTIC SEARCH USING COSINE SIMILARITY

Aim

To develop a semantic search system that converts a collection of documents and a user query into numerical vector embeddings, calculates cosine similarity between them, and retrieves documents in descending order of relevance.

Software Requirements

Python 3.x, scikit-learn, and an IDE such as Visual Studio Code. Install the library with: `pip install scikit-learn`.

Procedure

1. Create a Python file named semantic_search_cosine.py.
2. Import TfidfVectorizer and cosine_similarity from scikit-learn.
3. Prepare a list of sample documents that will be searched.
4. Enter a user query describing the information to retrieve.
5. Fit the TF-IDF vectorizer on the document collection to generate document vectors.
6. Transform the query using the same vectorizer to generate its query vector.
7. Calculate cosine similarity between the query vector and every document vector.
8. Sort the similarity scores in descending order and display the ranked documents.
9. Run the program and identify the document with the highest similarity score.

Program Code

```
# Semantic Search using Cosine Similarity
# TF-IDF is used to create document and query vector embeddings.

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 1. Sample document collection
documents = [
    "Machine learning allows computers to learn patterns from data.",
    "Artificial intelligence makes machines perform tasks that need human intelligence.",
    "Python is widely used for software development and data analysis.",
    "Solar energy is a renewable source of electricity.",
    "Deep learning uses multi-layer neural networks."
]

# 2. User search query
query = "How do computers learn using data?"

# 3. Generate document embeddings
vectorizer = TfidfVectorizer(stop_words="english")
document_embeddings = vectorizer.fit_transform(documents)

# 4. Generate query embedding using the same vectorizer
query_embedding = vectorizer.transform([query])

# 5. Calculate cosine similarity
similarity_scores = cosine_similarity(
    query_embedding, document_embeddings
).flatten()

# 6. Rank documents by similarity score (highest first)
ranked_results = similarity_scores.argsort()[::-1]

# 7. Display ranked search results
print("SEMANTIC SEARCH SYSTEM")
print("Query:", query)
print("\\nRanked Results:")

for rank, doc_index in enumerate(ranked_results, start=1):
    print(f"\\nRank {rank}")
    print(f"Similarity Score: {similarity_scores[doc_index]:.4f}")
    print(f"Document: {documents[doc_index]}")
```

Execution Screenshot

Output captured from executing the Python program:

Python Terminal | Semantic Search

```
SEMANTIC SEARCH SYSTEM
Query: How do computers learn using data?
\nRanked Results:
\nRank 1
Similarity Score: 0.6486
Document: Machine learning allows computers to learn patterns from data.
\nRank 2
Similarity Score: 0.1550
Document: Python is widely used for software development and data analysis.
\nRank 3
Similarity Score: 0.0000
Document: Deep learning uses multi-layer neural networks.
\nRank 4
Similarity Score: 0.0000
Document: Solar energy is a renewable source of electricity.
\nRank 5
Similarity Score: 0.0000
Document: Artificial intelligence makes machines perform tasks that need human intelligence.
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", li
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadshe
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in _warm_feature_
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in _warm_drawing_
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in brin
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element collection.

[Process finished with exit code 0]
```

Result

The semantic search program executed successfully. It generated vector representations for the documents and query, computed cosine similarity scores, and ranked the documents from most relevant to least relevant. The document with the highest score is presented as the top search result.

Technical Note: This beginner-friendly implementation uses TF-IDF vectors as embeddings. TF-IDF with cosine similarity is a vector-space information-retrieval baseline; it is not a contextual transformer embedding model. For deeper meaning-based matching across different wording, replace TF-IDF with sentence-transformers (for example, all-MiniLM-L6-v2) and retain cosine similarity for ranking.